PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

Facultad de Ciencias e Ingeniería

EVALUATION OF AN ALGORITHM FOR STAR IDENTIFICATION AND ATTITUDE DETERMINATION OF CUBESATS

Thesis to obtain the Professional Title of Electronic Engineer

AUTHOR:

Alvaro Aranibar Roque

ADVISOR:

Neils Edison Vilchez Lagos

Lima, November 2025

TABLE OF CONTENTS

Page

INTRODUCTION 1

CHAPTER

1. PROBLEM FRAMEWORK 3

1.1 Problem description 3
1.2 Thesis objectives 5

2. THEORETICAL FRAMEWORK 7

2.1 Satellite Attitude and Star Trackers 7
2.2 Operating Principle and Noise Considerations 9
2.3 Camera Imaging Parameters and Effects 11
2.4 Image Preprocessing 15

3. CONCEPTUAL DESIGN AND COMPONENT SELECTION 19

3.1 Requirements 19
3.2 System Block Diagram 19
3.3 Microcomputer Selection 20
3.4 Opto-electronic Components Selection 21

4. ALGORITHM DEVELOPMENT 25

4.1 Off-board look-up table generation 25
4.2 Thresholding and centroiding 31
4.3 Geometric transformations and catalog matching 31

5. EVALUATION AND RESULTS 33

5.1 NASA STEREO mission images 33
5.2 Distortion correction and camera parameter estimation 34
5.3 Preprocessing and star unit vector evaluation 37
5.4 Algorithm performance evaluation 40

CONCLUSIONS 42

FUTURE WORK 43

BIBLIOGRAPHY 45

INTRODUCTION

The rapid growth of CubeSats and other small satellite platforms has opened access to space for universities and emerging space programs, enabling missions that previously required large institutional budgets. Many of these missions rely on precise attitude determination and control in order to point optical payloads, maintain stable communication links, and perform scientific observations. Among the available sensors, star trackers stand out as absolute attitude sensors that deliver very high accuracy by matching stellar patterns in the captured images with a reference catalog, while gyroscopes and sun sensors provide only relative or coarse information and are affected by drift. However, commercial star trackers remain costly and often exceed the financial limits of academic or low budget missions, which motivates the development of alternative solutions based on commercial off the shelf components and open-source software.

In this context, this thesis proposes the implementation and evaluation of a star tracker algorithm for the INTISAT space mission. Two state-of-the-art lost in space star identification methods are selected and adapted: the SOST star tracker algorithm and the SVD Multilayer Voting algorithm. Both approaches address the star pattern recognition problem in the presence of measurement noise, optical distortions, and spurious sources such as cosmic rays, while keeping the memory footprint and processing time compatible with the constraints of a nanosatellite platform.

Requirements include limits on size, mass, power consumption, lens field of view, update rate, accuracy, and total cost, with a target hardware cost below 500 USD and typical commercial sensor costs on the order of 12000 USD for fine sun sensors and 40000 USD for

star trackers [1]. Processing pipeline integrates Source Extractor based centroiding, camera calibration, and Wahba based attitude estimation. The algorithms are validated using NASA STEREO HI 1 images and Astrometry.net solutions as reference, demonstrating the feasibility of a low-cost star tracker architecture suitable for future integration into the INTISAT attitude determination and control system.

CHAPTER 1

PROBLEM FRAMEWORK

This chapter analyzes the issues related to the need for an efficient star-tracking system for attitude determination in space missions. It describes the technical challenges posed by space environments and the limitations of current solutions. Finally, the objectives and scope of the study are presented.

1.1 Problem description

1.1.1 Need for COTS Hardware in CubeSats. Over the last decade, the development and deployment of CubeSats have intensified. Among their main advantages are shorter development times, the use of up-to-date technology due to their relatively short lifespan, smaller size, and, most importantly, greater accessibility for companies and universities thanks to their lower costs. While medium and large satellites weigh more than 500 kg, a 1U CubeSat corresponds to a cube with 10 cm sides and a mass of up to 2 kg. According to the CubeSat Design Specification, this size can be extended up to 12U [2]. Although they were initially developed to operate in Low Earth Orbit (LEO), at altitudes ranging from 350 km to 800 km above the Earth, space missions such as Mars Cube One (2018) have demonstrated their capability for interplanetary exploration [3].

Similarly, with the growing popularity of CubeSats, the use of COTS (Commercial Off-The-Shelf) hardware has also increased. This can significantly reduce the cost of space missions. For instance, the development and fabrication of a 1U CubeSat can cost around 30,000 USD [4], with launch costs reaching approximately 100,000 USD, in contrast to the 100,000,000 USD that a 100 kg satellite might cost [5]. These costs may vary depending on the mission.

Furthermore, components for the Attitude Determination and Control System (ADCS), such as fine sun sensors and star trackers, have average costs of approximately 12,000 USD and 40,000 USD, respectively [1].

1.1.2 Electronics in Space. In a study conducted on 855 CubeSats in 2019 [6], a mission failure rate of approximately 25% was estimated. Some of the main causes include abrupt temperature variations due to thermal shock (-65°C to 125°C), which occurs when transitioning from full sunlight to the Earth's shadow [7]; the “tin whisker” effect (see Fig. 1), which appears when modern lead-free electronics (required by RoHS regulations) are used in space missions and may cause short circuits; the non-hermetic sealing of plastic packaging that allows moisture to enter; the outgassing phenomenon; and exposure to radiation.

One of the vulnerabilities of COTS hardware used in space missions, such as microcontrollers, is the Single Event Upset (SEU), which consists of a change of state in internal transistors caused by ionizing particles. This can alter the programmed digital circuitry and lead to malfunctions known as soft errors.

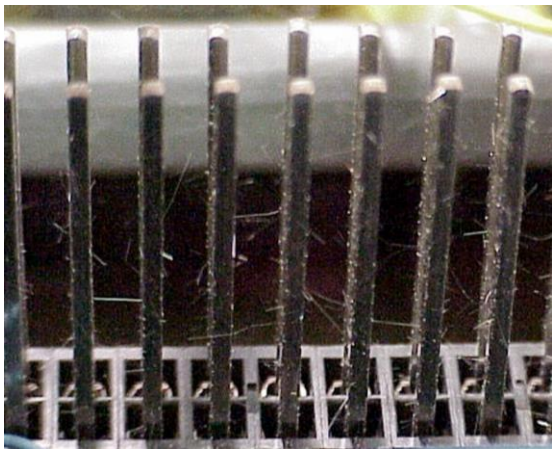


Fig. 1. “Tin whiskers” on pure tin connector pins after 10 years [8]

It is essential to implement measures to mitigate errors in space electronics. To prevent the “tin whisker” effect, tin-lead solder alloys with at least 3% lead or silver are effective [9]. Although radiation-hardened components can mitigate SEUs, their high-cost conflicts with the low-cost nature of CubeSats. A more economical approach is Radiation Hardening by Design (RHBD), which uses redundancy for error detection and correction. Additionally, shielding with metals such as aluminum, copper, lead, or tungsten protects against electrons and gamma rays, while hydrogen-rich plastics like polyethylene or polypropylene reduce proton and neutron impact. For instance, the Shields-1 CubeSat [10] achieved 30% higher radiation resistance using aluminum and Z-grade tantalum shielding compared to standard aluminum.

1.2 Thesis Objectives

Due to the limited budget for acquiring commercial attitude determination systems and the limited accuracy offered by other components such as sun sensors, this work proposes the implementation of a star tracker using a COTS microcomputer and camera. Unlike other attitude determination methods that rely heavily on hardware precision, the complexity of a star tracker lies primarily in its software and its robustness to noise.

1.2.1 General objective. To implement and evaluate a star tracker algorithm for the INTISAT space mission, intended for a 3U CubeSat operating in Low Earth Orbit (LEO).

1.2.2 Specific objectives

a) Define the requirements of the star tracker. These should specify the system's dimensions and weight, electrical power, processing time, resolution, and photographic system characteristics according to the limitations of a 3U CubeSat.

b) Develop the conceptual design of the star tracker, including its block diagram.

c) Evaluate and compare the electronic, optical, and photographic components from both technical and economic perspectives, in order to select the optimal solution.

d) Analyze different algorithms that address the Lost-in-Space problem and identify the two most robust alternatives against noise, with the best energy and computational efficiency.

e) To implement and assess the performance of the selected algorithm using satellite on-space images. The memory footprint, processing time, and boresight pointing accuracy must be determined.

CHAPTER 2

THEORETICAL FRAMEWORK

2.1 Satellite Attitude and Star Trackers

The attitude of a satellite refers to its angular orientation relative to a reference frame, such as its orbit, the stars, or other inertial objects. To describe this attitude, mathematical tools such as Euler angles and quaternions are used. As shown in Fig. 2, Euler angles define sequential rotations about three axes in a specific order, indicating how to transform from one reference frame to another. However, this technique can present difficulties, such as the gimbal lock phenomenon. In contrast, quaternions provide a more efficient and accurate way to represent rotation, describing it as a single rotation about an axis, which avoids the issues associated with Euler angles and reduces complexity to only four parameters.

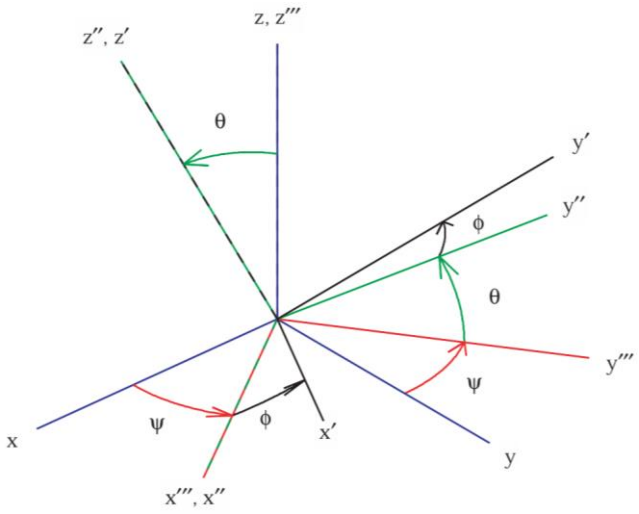


Fig. 2. Sequence of Euler angles [11]

To measure the attitude of a satellite, it is essential to define two reference frames: the orbital frame and the body frame. The orbital frame establishes the coordinates that describe the satellite's ideal orientation in its orbit, while the body frame is fixed to the satellite itself. The attitude is determined by comparing how the body frame is oriented with respect to the orbital

frame at a specific moment. This measurement is crucial for the satellite to perform its functions, such as accurately pointing toward an astronomical target or keeping its antennas aligned with Earth. Star trackers are ideal for these tasks, as they can precisely determine the attitude by comparing the positions of captured stars with a stored catalog, achieving very high levels of accuracy.

Star trackers stand out as absolute attitude sensors, meaning they provide a direct and precise measurement of a satellite's orientation in space. In contrast, other systems such as gyroscopes or inertial measurement units (IMUs) only detect relative changes in attitude, which can lead to error accumulation due to measurement noise. While these systems rely on an initial reference, star trackers correct such cumulative errors, maintaining precise orientation throughout the mission. Owing to their high accuracy, which can exceed 0.1° [12], they are indispensable in space missions where attitude control is critical. Furthermore, star trackers offer advantages over other methods, such as satellite navigation systems, since they operate independently. They can also determine a satellite's orientation without prior knowledge through the so-called "lost-in-space" attitude determination, allowing them to function autonomously without relying on previous references [12].

Despite their high precision, star trackers can be affected by factors such as sunlight reflections from the spacecraft or exhaust gases, as well as various optical errors like spherical or chromatic aberration. External interferences can also hinder the star identification process, including planets, comets, nearby satellites, or light pollution from large cities on Earth.

2.2 Operating Principle and Noise Considerations

The main objective of a star tracker is to process the images captured by the sensor to determine the centroid coordinates (X and Y) and the intensities of each detected star. These coordinates allow the recognition of stellar patterns based on a star catalog stored in memory.

The apparent magnitude of a star, used as a reference in catalogs, measures its brightness on an inverse logarithmic scale from the observer's perspective. This means that the lower the apparent magnitude value, the brighter the celestial body. For example, the Sun has an apparent magnitude of $Mv = -26.832$, while dimmer stars have higher magnitudes.

The transmission of images from the sensor to the processor occurs sequentially, pixel by pixel and line by line, even if the data bus is parallel. This configuration allows for real-time processing optimization. Furthermore, when working with monochromatic cameras, the brightness of each star is directly represented by the pixel value, which is related to the resolution of the sensor's analog-to-digital converter (ADC).

2.2.1 Noise Sources. In astronomical image capture, various sources of noise can affect data quality and the accuracy of the star tracker. One of the main sources is readout noise, which is introduced during the analog-to-digital conversion process and data transfer from the sensor. This type of noise includes quantization noise, related to the resolution of the analog-to-digital converter (ADC), and spurious electrons, which are random fluctuations generated within the sensor.

Another significant source is thermal noise, also known as dark current, which is caused by the heat generated in the electronic components of the sensor. Even in the absence of light, thermally generated electrons within the sensor can produce false signals that simulate

captured light. This effect increases with temperature and is particularly problematic in space, where vacuum conditions make heat dissipation difficult.

As shown in Fig. 3, dark current and readout noise generate white spots that uniformly cover the image. The white line appears as a result of damaged pixels [13]. In many systems, thermal noise can be mitigated through active cooling techniques or proper thermal design of the satellite.

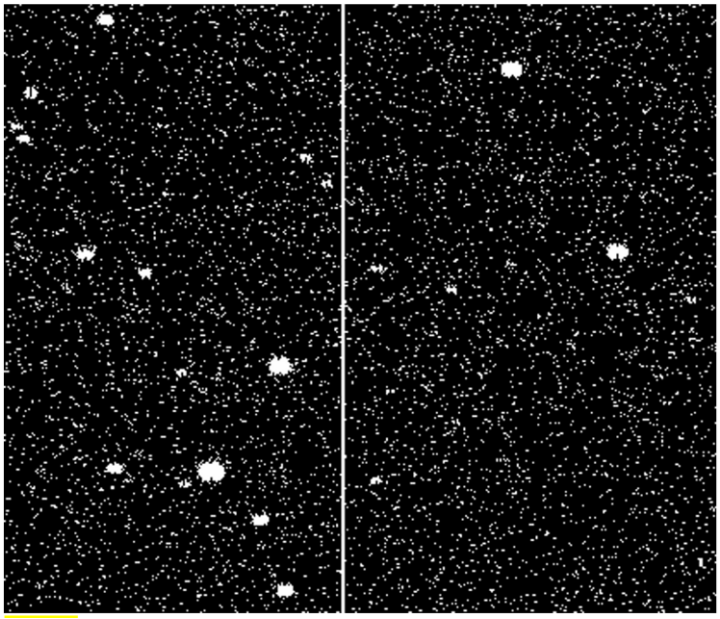


Fig. 3. Image with noise captured at the Tartu Observatory [13]

The brightness of other celestial bodies, such as the Moon or scattered light from Earth, can introduce unwanted artifacts in the captured images. These external light sources may interfere with star identification, creating spots or reflections that complicate image processing. For this reason, it is essential to minimize the impact of ambient light through the use of filters or exposure adjustments.

Finally, cosmic rays represent a unique source of noise in space environments. These high-energy particles can strike the sensor and saturate multiple pixels in an area, producing bright

patterns that do not correspond to real stars. In extreme cases, cosmic rays can cause permanent damage to the sensor's pixels, reducing its functionality over time.

2.2.2 Operating Modes of Star Trackers. Commercial star trackers typically operate in two main modes that adapt to different mission stages and conditions: Lost in Space (LIS) mode and Tracking Mode [14].

The LIS mode is used when no prior information about the satellite's attitude is available. This mode is computationally intensive since it requires digitizing and processing the entire image captured by the sensor to identify stellar patterns from scratch. The process involves detecting stars, calculating their centroids, and comparing them with a star catalog to determine the satellite's orientation. This mode is critical during system initialization or after disruptive events such as maneuvers or resets, when previous attitude information has been lost.

On the other hand, the Tracking Mode is more resource-efficient and is activated when the system already has prior attitude information. In this mode, the tracker uses the previously estimated attitude data to predict the positions of the star centroids in the current image. This eliminates the need to process the entire image, limiting the analysis to specific regions of interest. As a result, energy consumption and computational requirements are significantly reduced.

2.3 Camera imaging parameters and effects

2.3.1 Field of view (FOV). A convex lens focuses and forms an image by directing incoming light rays onto the focal plane, where the image sensor is located. As shown in Fig.

4, the focal length f is the distance between the optical center of the lens and the focal plane when the lens is focused at infinity.

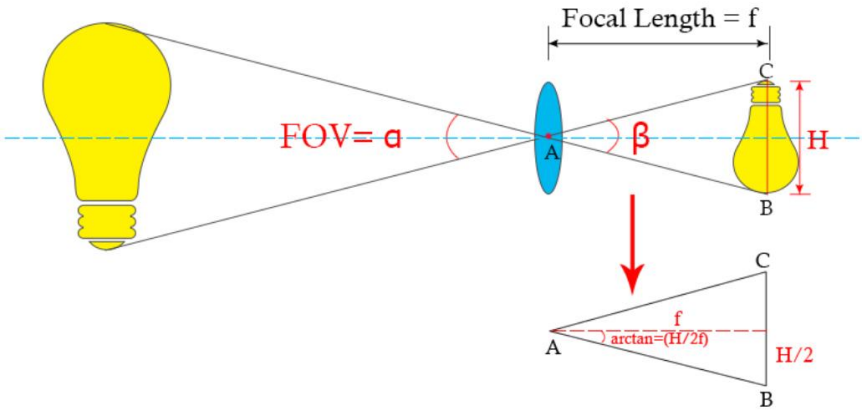


Fig. 4. Geometry of a camera system's FOV [15]

From geometric considerations, with the angle $\alpha = \beta$, the horizontal and vertical FOVs are obtained by applying (1) to the horizontal and vertical sensor dimensions, respectively.

$$FOV = 2 \cdot arctan(\frac{sensor\ size}{2 \cdot f}) \tag{1}$$

A larger sensor or a shorter focal length provides a wider FOV, capturing a greater portion of the scene. Conversely, a smaller sensor or a longer focal length reduces the FOV, producing the optical zoom effect, which results in an apparent magnification of the captured image.

2.3.2 Exposure. Refers to the amount of light that reaches the camera sensor during image capture and is expressed by (2).

$$\mathrm{H} = \mathrm{E} \cdot \mathrm{t} \tag{2}$$

where H is the luminous exposure, measured in lux·seconds (lx·s), and E is the illuminance on the sensor, which depends on the lens aperture (f-number). A larger aperture (smaller f-

number) allows more light to enter. The term t represents the exposure time, controlled by the shutter speed, which determines how long the shutter remains open.

Other factors also influence the image sensitivity indirectly. Larger pixels can capture more photons, increasing the signal and improving performance in low-light conditions. Likewise, a larger sensor collects more total light, reducing noise and enhancing overall image quality. The selection criteria for the optoelectronic system in Chapter 3 should take this dependencies into account.

2.3.3 Rolling shutter and motion blur effects. COTS-based microcomputers and camera sensors were chosen for three previously deployed satellites: Polar Satellite Launch Vehicle C-55 [16], SUCHAI-2 (a Chilean CubeSat swarm) [1], and OreSat 0.5 [17], Studies conducted after the satellite deployment [16] indicated failures for motion rates above 0.4°/s. In Fig. 5, an image shows elongated star trails as a result of tumbling. The effects of rolling shutter and motion blur are clearly visible.

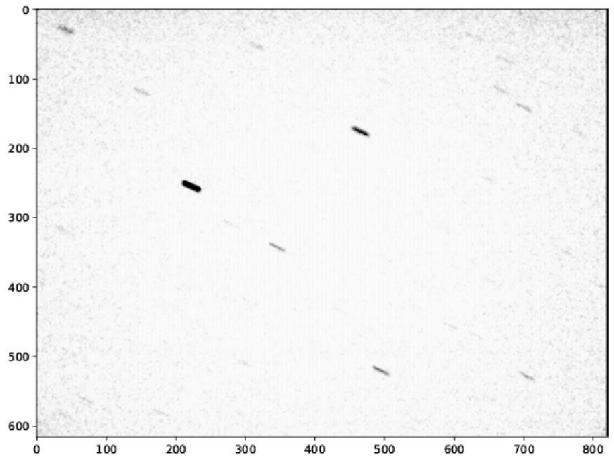


Fig. 5. Rolling shutter and motion blur effect in an image captured by mission [16]

Rolling shutter occurs in cameras where each row of the sensor is exposed at different times. If an object moves while the sensor is being read, its image can appear distorted or

"skewed," even if the overall exposure is correct. This effect is caused by the sensor's sequential readout method. This problem can be completely eliminated by using a global shutter, as shown in Fig. 6 (a), which exposes all rows simultaneously. Both [1] and [16] used a rolling shutter Raspberry Pi Camera V2, which caused the geometric patterns of the stars in the images to appear distorted, preventing their correct identification when the satellite was rotating.

Motion blur occurs when an object moves quickly during the exposure time, causing its image to appear blurred or stretched. It can occur in any camera, whether global or rolling shutter, as shown in Fig. 6 (b) and Fig. 6 (c), and depends on the object's speed. Motion blur can be mitigated by reducing the shutter speed, which is configurable on each camera, but care must be taken to avoid underexposure.

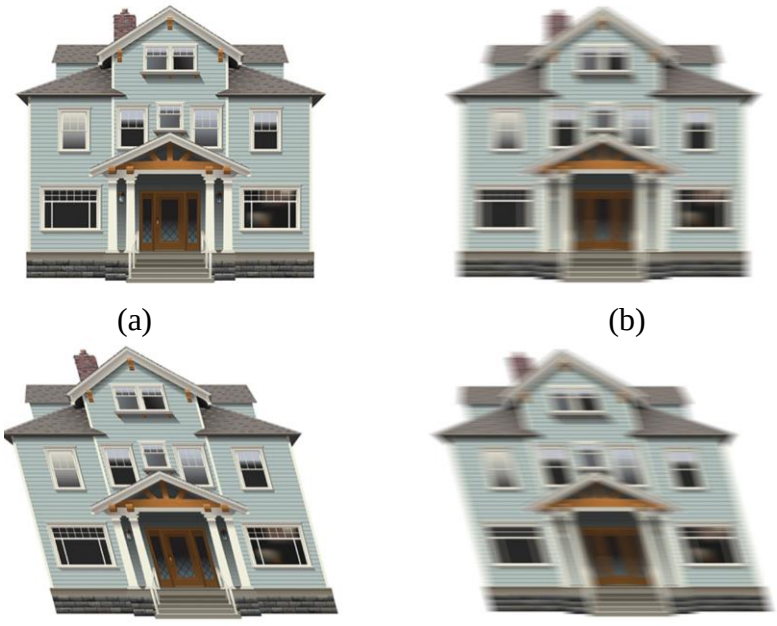


Fig. 6. Images with: (a) Global shutter camera (b) Global shutter camera with motion blur (c) Rolling shutter camera without motion blur (d) Rolling shutter camera with motion blur [18]

The velocity of a satellite in Low Earth Orbit (LEO) typically ranges between 6.9 km/s and 7.8 km/s relative to the Earth [19]. However, its relative motion with respect to the stars is very small since they are extremely distant. Therefore, in a CubeSat, the rolling shutter effect is more influenced by its rotation speed than by its translational motion.

2.4 Image Preprocessing

The following algorithms are presented to filter noise and determine the centroids of each star in an image as part of the preprocessing. The algorithms for geometric transformations and catalog matching will be detailed in Chapter 4.

2.4.1 Thresholding and Blob Detection. The goal is to segment the groups of bright pixels representing the stars, which will be referred to as blobs. A blob can be distinguished from the dark background by an abrupt change in pixel intensities. This can be quantified using the difference technique, which consists of subtracting the previous pixel value I_{j-1} from the current pixel value I_j, as shown in (3).

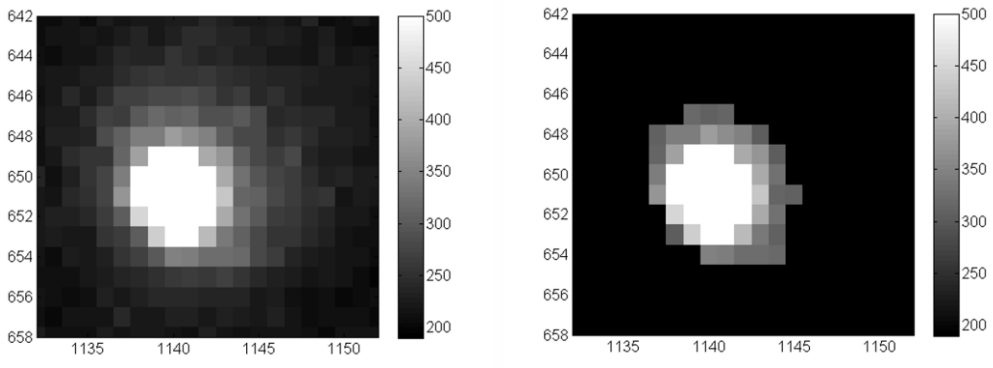
$$\partial I_j = \left| I_j - I_{j-1} \right| \tag{3}$$

In [20], this difference is compared with a fixed threshold that allows bright pixels to be distinguished from the background. If the difference is greater than the threshold, a new blob is detected. However, this method has the disadvantage of not adapting to changing backgrounds or an overall increase in image brightness, which can cause unwanted celestial objects to be detected. For example, the Moon can increase the average pixel intensity of the background, consequently reducing the difference ∂I_j. In this case, some stars might be inadvertently filtered out because the threshold is fixed and was previously defined under different lighting conditions.

On the other hand, since the profile of a star can be modeled as a Gaussian distribution [21], if the slopes are very small, faint stars might not be classified as blobs. This problem is addressed in [13] by determining a dynamic threshold, which is updated with each received pixel and set equal to five times the standard deviation (4) of the intensity I. This ensures that the probability of a noisy pixel exceeding the threshold, assuming the noise levels follow a normal distribution, is only 0.023%.

$$\sigma = \sqrt{\frac{1}{N}\sum_{j=1}^{N}(I - I_{promedio})} \tag{4}$$

In Fig. 7 (a), the original image of a star is shown, and in Fig. 7 (b), after thresholding, where the blob is separated from the background.



(a) (b)

Fig. 7. (a) Original image. (b) Thresholded image [20]

On the other hand, in an ideal situation, all the pixels of a star are consecutively bright. However, in reality, defective (dark) pixels may be received that do not exceed the threshold in the middle of a row intended to be classified as a blob. For this reason, in [20] it is proposed to establish a tolerance called the ghosting distance to prevent the pixels following a defective one from being undesirably classified as a new blob instead of remaining part of the current

blob. In Fig. 8, it is shown how a tolerance is set for the detection of a new blob with a ghosting distance equal to 3.

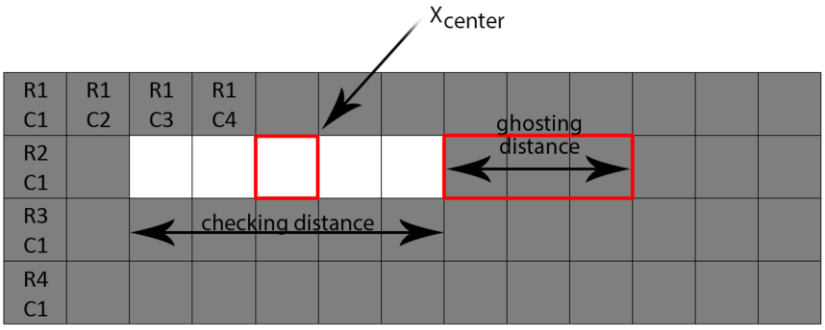


Fig. 8. Checking and ghosting distance for classifying a blob row [20]

2.4.2 Centroid Determination. The objective of the algorithm is to determine the centroids X_{blob} and Y_{blob} of each blob with respect to the first pixel received in the frame, as well as the average intensity $I_{b\,prom}$ of each one. Since the data is received pixel by pixel and row by row, it is convenient to initially calculate the centroid of each row X_{fila} as it is being received using (5).

$$X_{fila} = \frac{\sum_{k=1}^{N}(k \cdot I)}{\sum_{k=1}^{N}(I)} + X_{inicial} \tag{5}$$

The equation consists of an average of the position of each pixel weighted by its intensity relative to the total intensity of the row. In addition, the pixel position $X_{inicial}$ is added to obtain the absolute position of the centroid with respect to the first pixel of the frame, since k is counted from the first pixel of the identified blob.

Next, using (6), the centroid of the entire blob X_{blob} is determined by averaging the centroids of each row calculated previously.

$$X_{blob} = \frac{\sum_{f=1}^{F} X_{fila}}{F} \tag{6}$$

Where F is the total number of rows identified for each blob. Finally, in (7), the centroid Y_{blob} is determined.

$$Y_{blob} = \frac{\sum(I_{fila} \cdot c)}{\sum I_{fila}} + Y_{inicial} \quad (7)$$

Where c is the column count of the blob and I_{fila} is the sum of the pixel intensities of each row. Likewise, the total intensity of the blob I_{blob} has already been calculated using $\sum I_{fila}$ in the previous equation.

CHAPTER 3

CONCEPTUAL DESIGN AND COMPONENT SELECTION

3.1 Requirements

As this is an engineering model for a 3U CubeSat and based on the specifications of star trackers reported in the literature, the requirements are defined in Table 1.

Table 1. General requirements

Parameter	Value
Size (mm)	≤ 100x80x70
Weight (kg)	≤ 0.5
Power (W)	≤ 1.3
Accuracy (°)	≤ 0.008
Update rate (Hz)	≤ 0.3
Lens FOV (°)	≥ 10
COST (USD)	≤ 500

3.2 System block diagram

Fig. 9 shows the block diagram of the star tracker system. The Raspberry Pi is powered with 5V. The lenses focus the image and send it to the Raspberry Pi Zero 2 W through the MIPI interface, which also provides power to the camera. More details about the selection of the microcomputer and camera sensor are presented in the following sections. The Raspberry Pi processes the image on-board through a five-stage algorithm, from which the satellite attitude is obtained at the system output.

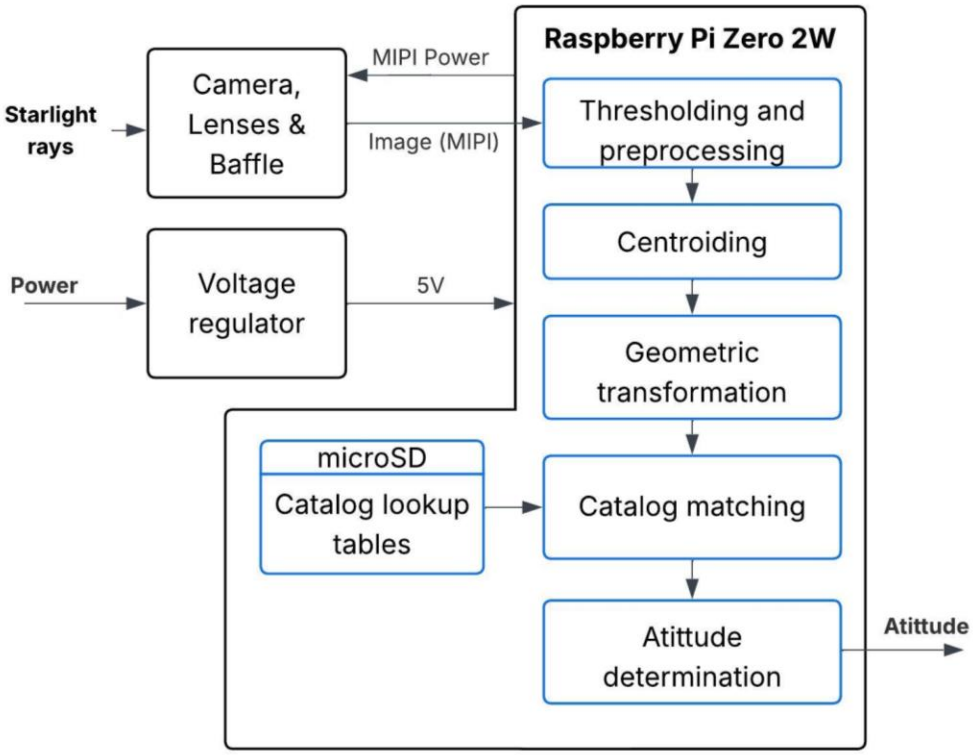


Fig. 9. Block diagram of the star tracker system

3.3 Microcomputer selection

The algorithms could be implemented in Linux based microcomputers. COTS-based microcomputers were chosen as the star tracker processing platform in three previously deployed satellites: Polar Satellite Launch Vehicle C-55 [16], SUCHAI-2 (a Chilean CubeSat swarm) [1], and OreSat 0.5 [17], using a Raspberry Pi Zero, a Raspberry Pi Zero 2 W and a PocketBeagle respectively. Table 2 shows a COTS microcomputer modules comparison. Due to its balance of performance and cost, achieved by integrating a quad-core CPU at a low price, the Raspberry Pi Zero 2W was selected. Likewise, thermal-vacuum, vibration and radiation effects tests were performed showing no signs of failure in [1] and [22].

Table 2. COTS microcomputer modules comparison

Characteristic	RPi Zero 2 W	Pocket Beagle 2	RPi 3B+	NanoPi Neo Air	ASUS Tinker board
CPU	1 GHz 64-bit quad-core Arm Cortex-A53	Quad-core Arm Cortex-A53 Up to 1.4 GHz	1.4 GHz 64 bit quad-core ARMv8	Quad-core Cortex-A7 Up to 1.2GHz	Quad-core 1.8 GHz ARM Cortex-A17
Memory	512 MB SDRAM	512MB LPDDR4	1 GB SDRAM	512 MB DDR3	2 GB DDR3
Spacial heritage	Yes	No	No	Yes	No
Power consumption (W)	1	1.25	2.5	2	4
Weight (g)	11	12.7	45	10	55
Average cost (USD)	10	29.29	44	23	159.99

3.4 Opto-electronic components selection.

Many current algorithms fail when only a few stars are detected within the FOV [23]. In this context, the number of detected stars depends on the characteristics of the sensor and the lenses, which determine the sensitivity, exposure, and field of view (FOV).

3.4.1 Camera module selection. There are two main technologies for image sensors: CCD and CMOS. The latter have become increasingly popular in recent space missions due to their lower power consumption, reduced cost, smaller size, higher readout speed, and technological advancement. Although CCD sensors provide higher fidelity in detail and color, this characteristic is not critical in this context, as the image processing focuses on the detection of bright spots (blobs) in the captured images. Table 3 compares four selected camera modules, two of which have space heritage. The modules were chosen considering their performance in low-light environments, availability, and cost.

Table 3. Camera modules comparison

Element	RPi camera V2.1	Arducam AR0134	Lumenera Lw230M	RPi Global Shutter Camera
Sensor type	SONY IMX219PQ CMOS	ONSemi AR0134	Sony ICX274 CCD	Sony IMX296LQ R-C
Resolution (Mpix)	8	1.2	2	1.58
Diagonal sensor size (mm)	4.60	6	8.923	6.3
Pixel size (µm)	1.12	3.75	4.40	3.45
Shutter	Rolling	Global	CCD	Global
Spacial heritage	Yes [1] [16]	Yes [17]	No*	No
Weight (g)	3	21	300	41
Power (W)	0.5	0.16	4	-
Average cost (USD)	16.50	88.39	1950	55

*Part of the low cost star tracker license from Nasa Technology Transfer Program [24]

- **Resolution and pixel size:** Higher resolution allows distinguishing stars that are close to each other and capturing more detailed images, which is important for accurately identifying star patterns. However, it also has drawbacks, as higher resolution increases the data volume per image, thereby raising storage and processing requirements, which are limited in CubeSats. In addition, increasing resolution usually comes with a reduction in pixel size. A larger pixel size is desirable because it improves the sensor's ability to collect light under low-illumination conditions, as mentioned in Chapter 2**.**
- **Shutter:** As mentioned in Section 2.3, the use of rolling shutter cameras was one of the causes that prevented successful star identification at rotation rates above 0.4°/s. Therefore, global shutter or CCD cameras are better alternatives, since they provide simultaneous pixel exposure.

- **Sensor size:** A larger sensor is preferred, as it allows a wider Field of View (FOV).
- **Communication protocols:** Since the Raspberry Pi Zero 2 W features a MIPI CSI-2 connector, sensors with the same interface are proposed for easier integration. This interface also allows configuring camera parameters such as exposure time and module power supply.

Balancing the considerations mentioned above, the RPi Global Shutter Camera was selected. However, most global shutter sensors have limited resolution. To avoid reducing the star tracker's precision, this limitation will be compensated by using lenses with a higher zoom (smaller field of view, FOV).

3.4.2 Lenses selection. To mitigate the motion blur effect, the exposure time should be reduced. This parameter can be configured through software. However, to compensate for the decrease in image brightness caused by a shorter exposure time, and according to equation (2), a lens with a larger aperture (smaller f-number) is preferred.

To compensate for the low resolution of the Raspberry Pi Global Shutter Camera, a zoom effect and, consequently, a smaller Field of View (FOV) is desired. As shown in [25], the FOV of commercial star trackers for small spacecraft typically ranges between 10° and 20°, which can be considered telephoto (narrow FOV). Nevertheless, within this range, a larger FOV is preferred since it increases the probability of detecting more stars.

Fig. 10 shows the lens that meets these requirements.

Fig. 10. PT3611614M10MP 16mm, 1:1.4 lens

To determine the FOV, the height (h) and width (w) of the sensor must be known. The sensor has an active resolution of 1456 × 1088 pixels, resulting in the ratio given by equation (8).

$$w = \frac{1456}{1088} \cdot h \tag{8}$$

Using the sensor diagonal (d) of 6.3 mm in equation (9), a width w = 5.047 mm and height h = 3.771 mm are obtained:

$$d^2 = h^2 + w^2 \tag{9}$$

By substituting the focal length of 16 mm and the sensor dimensions into equation (1), a horizontal FOV_H of 17.926° and a vertical FOV_V of 13.442° are obtained.

CHAPTER 4

ALGORITHM DEVELOPMENT

In this chapter, the algorithm for database generation and the on-board algorithm are described. On the one hand, it is not important for the former to be optimized, since it will be executed off-board on a desktop computer to generate the catalogs and lookup tables that will be stored in the memory of the microcomputer. On the other hand, the latter is intended to be executed on-board in a system with limited energy and computational power. Fig. 11 shows the general on-board algorithm flowchart. Each stage will be explained in detail in the following sections.

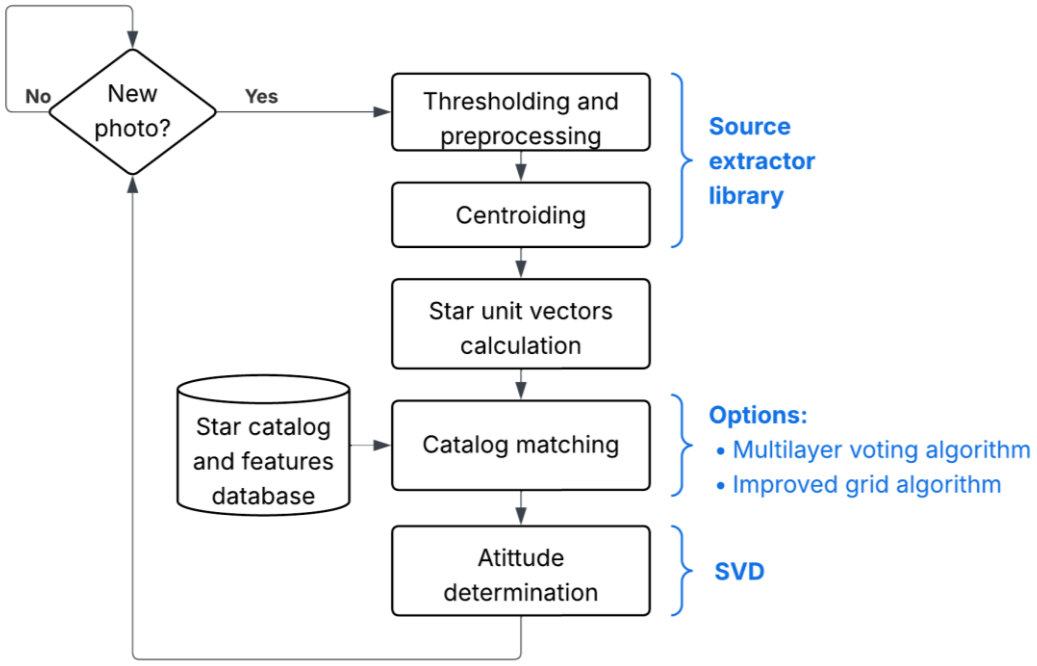


Fig. 11. General on-board algorithm flowchart

4.1 Off-board lookup generation

The VizieR astronomical catalog service is a database from which most existing catalogs can be downloaded. For this thesis, the SAO J2000.0 catalog will be used, which

contains 258,997 stars. The original catalog includes various labels providing astronomical information. Of these, only the following are relevant for this work:

- **delFlag:** Deletion flag; marks stars to be ignored if deleted.
- **Vmag:** Visual magnitude of the star (apparent brightness).
- **Rem:** Remarks, including duplicity and variability of the star.
- **RA.icrs (RA2000):** Right Ascension in the J2000.0 ICRS coordinate system (hours, minutes, seconds).
- **DE.icrs (DE2000):** Declination in the J2000.0 ICRS coordinate system (degrees, arcminutes, arcseconds).

After removing stars flagged by delFlag, variable and binary stars, and stars brighter than a threshold of 6 Mv, a total of 4355 stars remain. The brightness threshold can be adjusted according to the minimum apparent magnitude of the stars that can be imaged by the specific camera sensor to be used; in this case, the reference was the sensor used in [1] . At this stage, the number of stars in the catalog was significantly reduced.

An additional filter is applied because the catalog flags only indicate true double or binary stars, which are gravitationally bound. However, there are also visual or apparent double stars, which appear close together in an image simply due to alignment by chance. To address this, unit vectors for each star are computed, and stars whose angular separation (calculated from the inner product between their vectors) is less than 0.5° are removed.

Finally, 4286 stars remain, which are represented on the celestial sphere in Fig. 12.

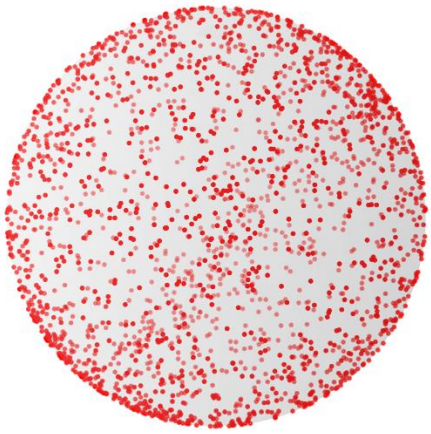


Fig. 12. Filtered SAO stars

. Fig. 13 shows an overview of the off-board lookup table generation process. In Algorithm 1 (SOST), the catalog is divided into patches/groups in 2D coordinates (a tangential plane projected onto the celestial sphere). In Algorithm 2 (Multilayer Voting), the patches/groups are generated in 3D.

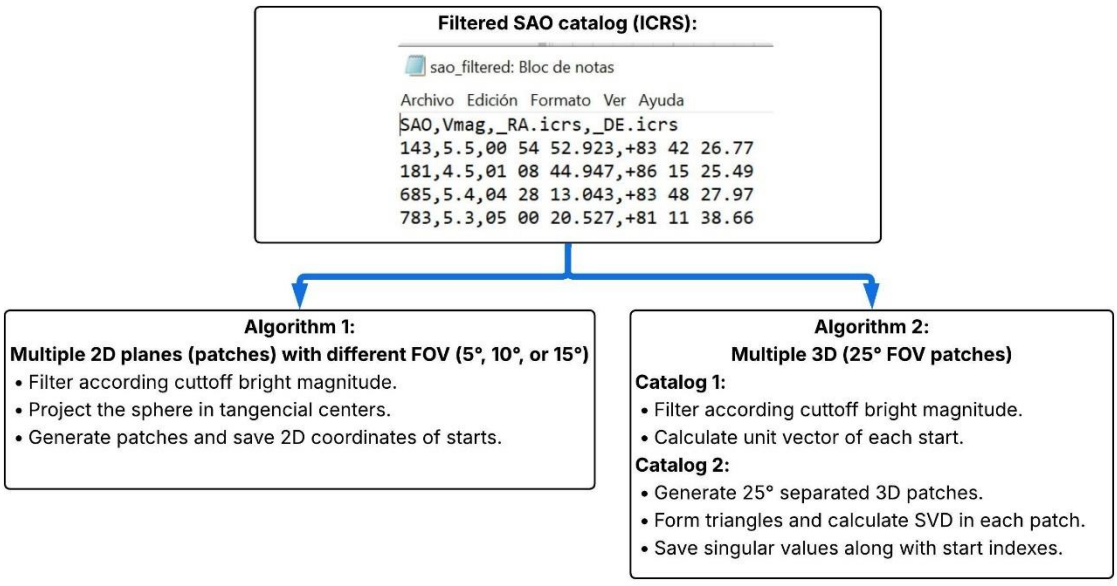


Fig. 13. Overview of Off-board lookup table generation

- **SOST algorithm:** The celestial coordinates from the star catalog (RA and DEC) are projected onto a 2D geometry using equations (10) from Textbook on Spherical Astronomy. This projection maps the celestial sphere onto a tangent plane, allowing it to be directly compared with the captured image. In this way, the star catalog is divided into multiple 2D planes (patches) with fields of view (FOV) of 5°, 10°,or 15°. The projection depends on the celestial coordinates of the sensor's pointing direction (the focal axis, with the image center as the projection point).

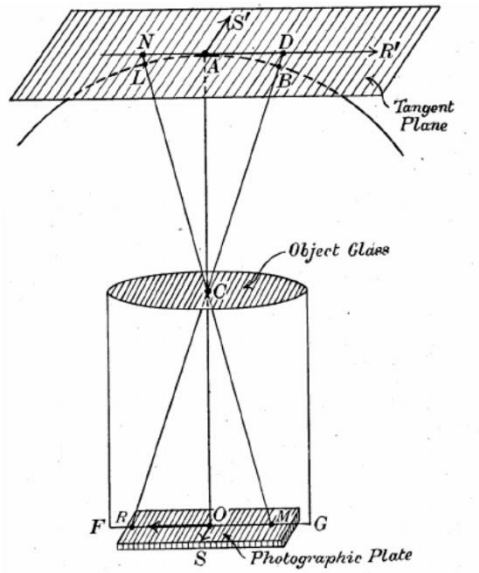


Fig. 14. Projection of celestial coordinates on tangent plane

$$\eta = \frac{\cos(\mathrm{DEC}) - \cot(\delta)\sin(\mathrm{DEC})\cos(\alpha - \mathrm{RA})}{\sin(\mathrm{DEC}) + \cot(\delta)\cos(\mathrm{DEC})\cos(\alpha - \mathrm{RA})},$$

$$\xi = \frac{\cot(\delta)\sin(\alpha - \mathrm{RA})}{\sin(\mathrm{DEC}) + \cot(\delta)\cos(\mathrm{DEC})\cos(\alpha - \mathrm{RA})}, \tag{10}$$

- **SVD multilayer voting algorithm:** Unlike the SOST algorithm, this one compares 3D characteristics of triangles (See Fig. 15) represented by their singular values (SVD). Two

lookup tables must be generated from the previously filtered catalog and stored in memory. A total of 187,963 triangles are generated.

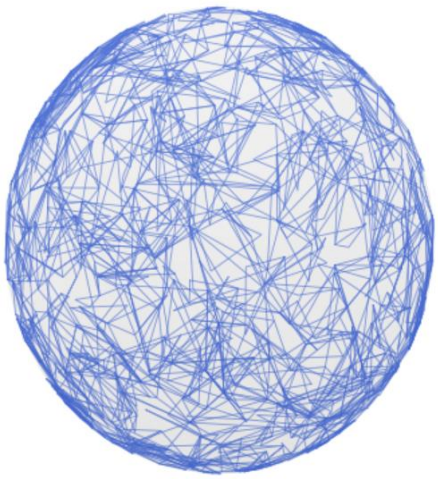


Fig. 15. Sample of generated triangles

The first lookup table contains the indices of each star and their respective unit vectors, calculated from the Right Ascension (RA) and Declination (DEC) of each star in the catalog using (11).

$$p_t^c = [\cos(\alpha_t)\cos(\delta_t)\ \ \sin(\alpha_t)\cos(\delta_t)\ \ \sin(\delta_t)]^T \tag{11}$$

The second lookup table is obtained after dividing the catalog into patches of 25° Field of View (FOV). 100,000 boresight vectors are generated using the Fibonacci grid method described in [26]. Fig. 16 shows an example of the algorithm applied to 700 boresights.

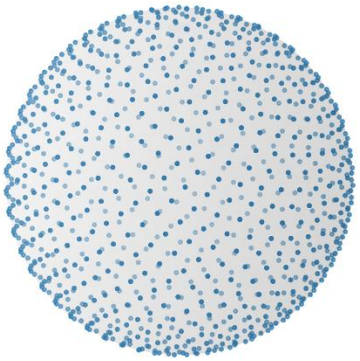


Fig. 16. 700 equally spaced boresights in the celestial sphere

Each boresight direction serves as the central point of a patch and acts as a reference for locating nearby stars. Within each patch, all possible triangle combinations are generated, as shown in Fig. 17. To avoid creating an excessively large catalog, only the 10 brightest stars in each patch are used. Additionally, it is ensured that no duplicate triangles are stored in the catalog.

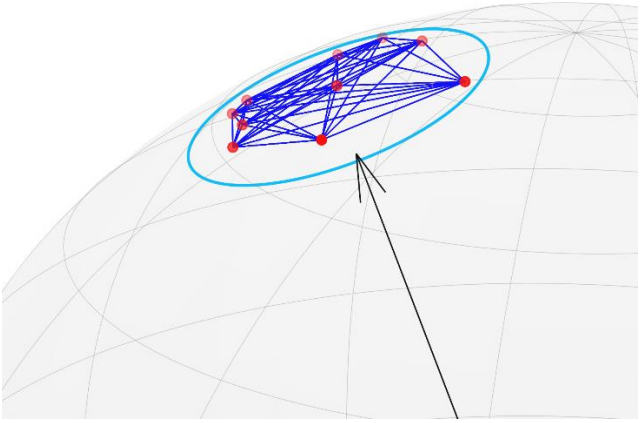


Fig. 17. Circular FOV with 10 stars triangle combinations

For each triangle, a 3×3 matrix is formed whose columns correspond to the unit vectors of the stars that compose it as shown in (12).

$$M_c = \begin{bmatrix} p_i^c & p_j^c & p_k^c \end{bmatrix} \tag{12}$$

From each matrix, the singular values are extracted, which contain information about the geometry and are also reference-frame independent parameters, making them more robust to noise. These singular values are then stored in the second lookup table, along with the indices of the stars forming each triangle.

4.2 Thresholding and centroiding.

For thresholding and centroiding, the widely used Python-based astronomy library Source Extractor is employed. Source Extractor is a program that extracts the pixel position and brightness of stars and galaxies in an image using neural networks. In the developed Python script, its execution was automated so that it returns the x and y coordinates of the 10 brightest stars in an image.

4.3 Geometric transformations and catalog matching

Each algorithm uses a different approach:

- **SOST algorithm:** Python-based Match library is used to compare the captured image with all patches, returning matrices that indicate geometric similarity. These matrices are used to refine the projection points of the patches over up to three iterations, ultimately identifying the patch toward which the sensor is pointing with high precision.
- **SVD Multilayer Voting Algorithm:** On the other hand, the onboard algorithm computes the unit vectors of the stars in the camera's reference frame using (8), where f is the focal length of the camera, and xy, yt zy the previously calculated centroid of each star:

$$p_t^b = \frac{1}{\sqrt{x_t{}^2 + y_t{}^2 + f^2}} [\ x_t \quad y_t \quad f\ ]^T \cdot (\mathrm{t} = \mathrm{i},\ \mathrm{j},\ \mathrm{k}) \tag{13}$$

Due to the parallax effect in astronomy, these vectors can be compared with those from the star catalog, since only distant stars were considered. Finally, similarly to the lookup tables of triangles previously created, 3×3 matrices are formed with columns containing the

unit vectors of each vertex of each possible triangle in the image and the singular values of each matrix are computed and compared with those in the catalog. Each time a match is found, the counters of the candidate stars are incremented. The stars with the highest number of counts are then selected as the final matches.

It is assumed that this algorithm would perform better than the one implemented on a Raspberry Pi Zero 2W onboard the Polar Satellite Launch Vehicle C-55 [16], since the previous algorithm compared only a single geometric feature (the angle between stars), which is more sensitive to noise according to previous studies.

CHAPTER 5

EVALUATION AND RESULTS

5.1 NASA STEREO mission images

There is still no universally accepted procedure for evaluating the performance of a star tracker. Some authors propose laboratory setups where a synthetic star field is displayed on a monitor and observed through collimation lenses in order to simulate stars at infinity [27]. This approach is convenient, but it has important limitations. First, standard monitors cannot reproduce the low brightness of faint stars with visual magnitude around 5. Second, mapping the spherical night sky onto a flat display introduces geometric distortions that complicate an accurate angular error analysis.

In this thesis, the algorithm is instead evaluated using real astronomical images from the NASA STEREO (Solar TErrestrial RElations Observatory) mission, following a similar approach to [1]. STEREO consists of two nearly identical spacecraft in heliocentric orbit, one leading and one trailing Earth, equipped with several optical instruments for heliospheric imaging.

For our experiments, we use images from the HI-1 L0 telescope [28]. This instrument has been operating since 2006 and has produced thousands of images containing rich star fields. Each image is stored in FITS format, and the associated metadata can be inspected with a FITS viewer such as Fv, as illustrated in Fig. 18.

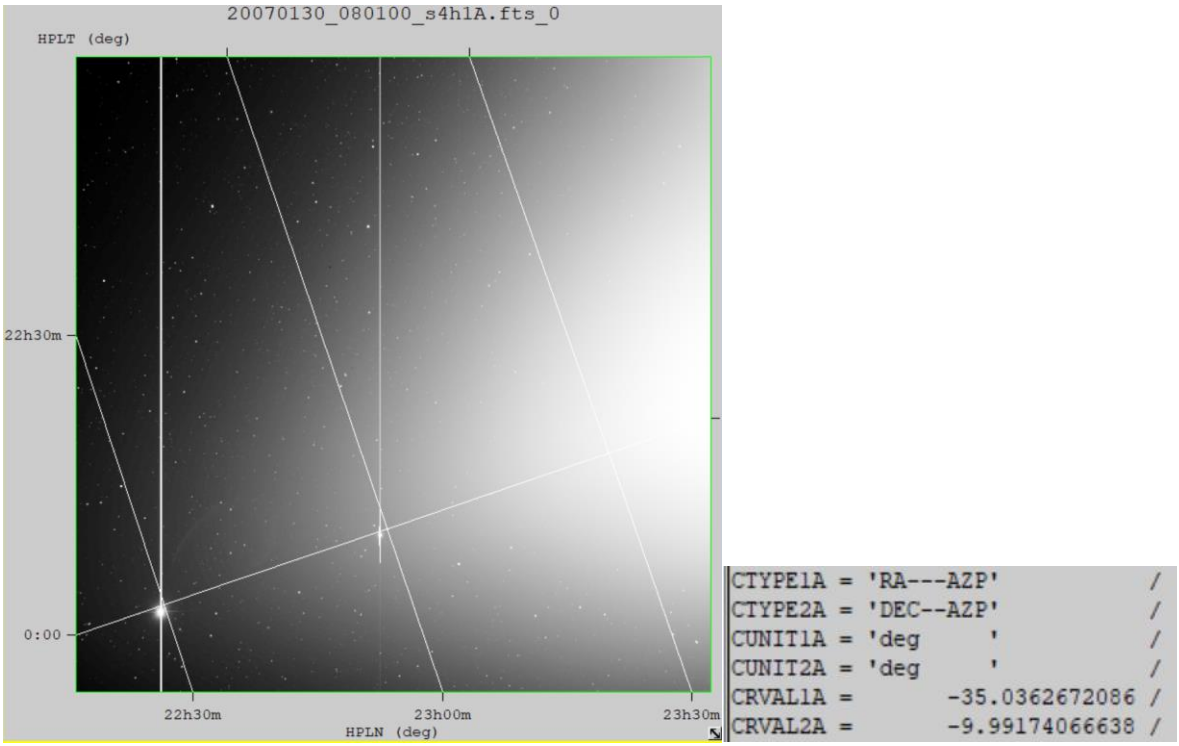


Fig. 18. Example of a STEREO HI-1 FITS image and its header metadata.

Among the many header keywords, those related to the World Coordinate System (WCS) are particularly important. They allow the conversion from pixel coordinates in the image to celestial coordinates in the ICRS frame. In particular, the keywords CRVAL1A and CRVAL2A provide the right ascension and declination of the camera boresight. The boresight is collinear with the optical axis and with the central pixel of the detector. The units of these angles are specified by CUNIT1A and CUNIT2A, which indicate that they are expressed in degrees. These WCS pointing values are used as ground truth when evaluating the attitude solution delivered by the star tracker algorithm.

5.2 Distortion correction and camera parameter estimation

As discussed in Chapter 1, converting pixel coordinates into 3D unit vectors in the camera frame requires an accurate camera model. The dominant source of error in angular measurements between stars is radial distortion, which modifies the apparent separation

between centroids on the image plane. If it is not properly corrected, the resulting bearing vectors are biased and the pattern-matching stage of the star identification algorithm can produce incorrect matches.

In this work two alternative calibration and projection models are considered. Section 5.2.1 is used throughout the thesis when working with STEREO HI-1 images, because WCS solution is available in the FITS headers. Section 5.2.2 describes a more conventional Brown–Conrady model combined with an intrinsic camera matrix, obtained from a dedicated calibration with a ChArUco target, as recommended in the COTS Star Tracker from NASA Johnson Space Center [29]. This method is better suited when calibrating a specific camera for a real hardware implementation

5.2.1 SIP convention coefficients and WCS parameters. Following the approach in [22], the distortion of the HI-1 telescope is modeled with the SIP (Simple Imaging Polynomial) convention. In this framework, astrometric calibration is first performed with Astrometry.net, which produces a WCS solution that includes polynomial distortion coefficients. These SIP coefficients are later used on board to correct centroid positions before they are converted into unit vectors.

Let *(u,v)* denote the centroid of a detected star in the pixel coordinate system, expressed relative to the SIP reference point (CRPIX1,CRPIX2). The distortion-corrected coordinates *(x,y)* are given by the SIP model in (14).

$$x = u + \sum_{a,b} A_{ab} u^a v^b$$
$$y = v + \sum_{a,b} B_{ab} u^a v^b \tag{14}$$

Where A_{ab} and B_{ab} are the SIP polynomial coefficients, typically up to second order, and the indices a and b are non-negative integers with $a+b \geq 2$. In practice only a small set of coefficients is nonzero and these are read directly from the FITS header.

Once distortion has been corrected, the algorithm converts *(x,y)* into angular offsets from the image center. Let (X_c, Y_c)denote the coordinates of the optical axis (usually close to the central pixel), and let *s* be the plate scale in radians per pixel, obtained from the WCS plate scale in arcseconds per pixel. The angular coordinates *(θx, θy)* of the star with respect to the boresight are approximated in (15).

$$\theta_x = (x - X_c) \cdot s \ , \ \theta_y = (y - Y_c) \cdot s \tag{15}$$

These angles represent small rotations around the camera *y* and *x* axes respectively. Assuming a pinhole model with the boresight aligned with the camera *z* axis, the corresponding bearing unit vector u_i associated with a star centroid is obtained (16).

$$u_i = \frac{1}{\sqrt{{\theta_x}^2 + {\theta_y}^2 + 1}} \begin{bmatrix} \theta_x \\ \theta_y \\ 1 \end{bmatrix} \tag{16}$$

5.2.2 Brown–Conrady distortion parameters and intrinsic camera matrix. For cameras that do not have a preexisting WCS solution, a dedicated geometric calibration is needed. The COTS Star Tracker documentation [30] recommends using a ChArUco target and the *charuco_cam_cal.py* tool provided with the software. The calibration consists of capturing several images of a flat ChArUco board so that the pattern covers the full field of view from different positions. These images are placed in the calibration directory and processed with the *charuco_cam_cal.py* script, which detects the ChArUco corners, performs the camera calibration, and returns both the intrinsic camera matrix and the Brown–Conrady distortion

parameters described in Chapter 2. A complete step-by-step description of this process is given in [30].

Unlike [30], where distortion correction is applied to the entire image, in this work it is proposed to apply it only to the centroids of the detected stars in order to improve computational efficiency. Each centroid *(u,v)* would first be corrected using the Brown–Conrady distortion model. In practice, this would be done with the OpenCV function undistortPoints, which returns the normalized coordinates *(xi',yi')* from which the bearing vectors can then be computed as in (16).

5.3 Preprocessing and star unit vector evaluation.

Before evaluating the star identification algorithms proposed in this thesis, it is necessary to verify that the camera parameters, the centroiding stage, and the conversion from pixel coordinates to star unit vectors in the camera body frame are all sufficiently accurate. If any of these steps is biased, the performance of the identification algorithms will be limited regardless of their theoretical robustness. For clarity, the evaluation is divided into two parts.

5.3.1. Verification of the SExtractor preprocessing. Astrometry.net is an astrometric solving tool that, in addition to providing calibration data, is able to identify stars and determine the boresight direction of an image in a blind manner, that is, without any prior information about its pointing. It has been used as an independent reference to assess the accuracy of star trackers in several works such as [22].

For each STEREO image, Astrometry.net produces a file named *corr.fits* that contains a table with the stars that were successfully matched to a reference catalog. Each

row includes, among other fields, the pixel coordinates of the detected star *(field_x, field_y)*, its equatorial coordinates in the ICRS frame *(index_ra, index_dec)*, and its measured flux. The flux value is not a calibrated apparent magnitude, but it is monotonically related to brightness, so higher flux corresponds to a brighter star (lower apparent magnitude).

To validate the preprocessing performed in this thesis, several STEREO images were processed with both Astrometry.net and the proposed pipeline based on SExtractor. For each image, the 15 brightest stars according to Astrometry.net (highest FLUX) were selected, as well as the 15 brightest stars according to SExtractor (lowest instrumental magnitude). A Python script then matched the two lists in pixel space.

Across the images tested, more than 10 matches per image were typically obtained between the two sources, which is more than sufficient for the subsequent identification algorithms. An example of this comparison is shown in Fig. 19, where the stars identified by Astrometry.net and SExtractor are overlaid on the same FITS image. To improve visualization, contrast enhancement filters were applied to the image using the Astropy library for Python.

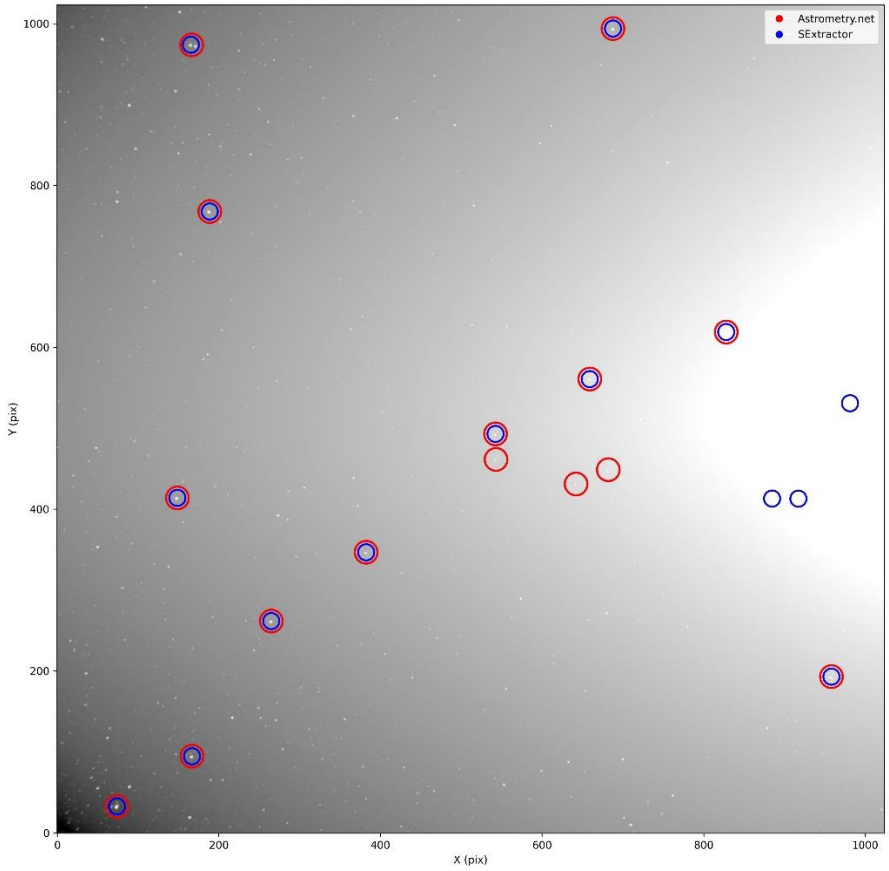


Fig. 19. Preprocessing comparison between Source Extractor and Astrometry.net

5.3.2 Verification of the bearing vectors. The next step is to validate the computation of the unit vectors used as inputs to the attitude estimation. For each matched star, two 3D unit vectors are available:

- A body-frame vector, obtained from the centroid in pixel coordinates using the camera model or WCS-based model described in Section 5.2, which represents the line of sight of the star in the camera frame.
- A reference-frame vector of the real stars, obtained by converting the catalog right ascension and declination (index_ra, index_dec) from Astrometry.net into a unit vector in the ICRS frame.

Forming matrices with these unit vectors as columns, Wahba's problem is solved via a singular value decomposition formulation.

Once the optimal rotation matrix is obtained, the camera boresight direction is propagated to the ICRS frame by applying the inverse rotation to the nominal optical axis of the camera $[0 \quad 0 \quad 1]^T$. The resulting boresight right ascension and declination are then compared with the reference pointing given by the FITS WCS header (Section 5.1). The angular separation between both directions is 0.5° on average, which confirms that the conversion from centroids to unit vectors is adequate for the subsequent evaluation of the star identification algorithms.

5.4. Algorithm performance evaluation

5.4.1. Pointing accuracy evaluation. A set of one hundred random images from the HI-1 L0 telescope, acquired between 2010 and 2015, was selected for the pointing accuracy assessment. For each image, the camera boresight vector in the body-fixed frame was transformed into the ICRS frame using the inverse of the rotation matrix (attitude matrix) K obtained from Wahba's solution, as shown in equation (17).

$$\overline{p}_{ICRS} = K^{-1}[0 \quad 0 \quad 1]^T \tag{17}$$

The resulting boresight vector $\overline{p}_{ICRS}$ was then converted to right ascension and declination and compared with the true pointing direction provided in the FITS header of each image. Fig. 20 shows the histogram of the resulting pointing errors. The distribution is centered around 0.55 degrees, and the maximum observed error is 0.75 degrees.

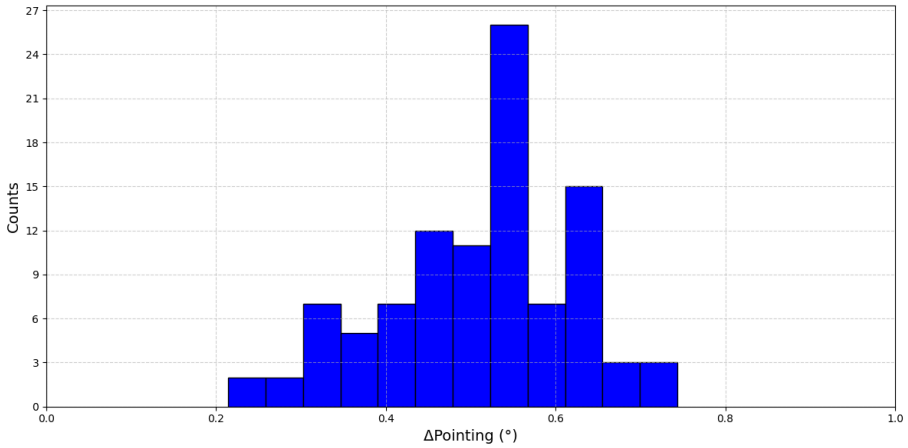


Fig. 20. Histogram of the algorithm's pointing-axis error.

5.4.2. Memory footprint and processing time evaluation. Table 4 presents the memory footprint of each file in the star catalog database.

Table 4. Memory footprint

File	Description	Size
C.npy	SAO indexes and unit vectors	108KB
ID.npy	Triangle indexes	2.92MB
LT_v1.pkl	Triangles with sing. val. V1	1.11MB
LT_v2.pkl	Triangles with sing. val. V2	1.12MB
	Total	5.25MB

Table 5 reports the average execution time of each function in the algorithm in an AMD Ryzen 5000 computer.

Table 5. Processing time

Function	Time
apply_sextractor	200 ms
centroid_to_unit_vectors	3 ms
optimized_grid_algorithm	84 ms
wahba_svd	72 ms
Total	0.359 s

CONCLUSIONS

This thesis presented the implementation and evaluation of a low-cost star tracker algorithm based for the INTISAT 3U CubeSat mission. From a survey of attitude determination techniques and commercial sensors, a set of requirements was derived that constrained size, mass, power, field of view, accuracy, update rate, and total cost. Based on these constraints, a complete concept was developed that includes both the algorithmic flow from image acquisition to attitude estimation and a concrete hardware platform proposed for implementation: a Raspberry Pi Zero 2 W, a Raspberry Pi Global Shutter Camera, and a 16 mm f over 1.4 lens. This configuration increases the number of detectable stars in a narrow field of view while preserving a sufficient signal to noise ratio at short exposure times, which is essential to limit motion blur in a tumbling CubeSat.

On the algorithmic side, two star identification methods were implemented and evaluated: the SOST algorithm, which operates on 2D projections of the celestial sphere divided into patches, and the SVD Multilayer Voting algorithm, which uses 3D triangle based features extracted from a filtered SAO J2000.0 catalog. A compact catalog database with a total memory footprint of about 5.25 MB was generated to fit within the resources of the proposed hardware. A preprocessing and calibration pipeline using Source Extractor, WCS or Brown Conrady camera models, and a Wahba SVD solution was validated with real NASA STEREO HI 1 images. The resulting pointing axis error distribution is centered around approximately 0.55 degrees, with a maximum of about 0.75 degrees, and an average processing time of 0.359 seconds per image on a desktop platform. Although the current performance does not yet reach the most stringent accuracy target, the results demonstrate

that a COTS based hardware and software architecture is technically feasible for a CubeSat class star tracker and provide a solid basis for a future on-board implementation.

FUTURE WORK

Future work will focus first on porting and optimizing the complete processing chain to the Raspberry Pi Zero 2 W, including centroiding, catalog lookup, and attitude estimation. Profiling on the target hardware should guide optimizations in C or C++ and the possible use of multi core processing to achieve update rates close to the 0.3 Hz requirement under realistic on board constraints. A dedicated calibration campaign is also planned for the selected global shutter camera and lens using a ChArUco target, following the Brown Conrady model and applying distortion correction directly to star centroids, with repeated calibration after thermal vacuum and vibration tests to verify parameter stability.

The robustness of the preprocessing stage can be improved by exploring alternative thresholding and blob detection strategies, adaptive background estimation, explicit cosmic ray rejection, and lighter centroiding methods tailored to the noise characteristics of the selected sensor. On the catalog side, additional work is needed to refine the tradeoff between catalog density, triangle configuration, and robustness to false detections, for example by tuning magnitude limits and geometric constraints according to the actual limiting magnitude of the flight hardware. Beyond catalog-based tests, extensive validation with planned on-sky tests that will capture real night sky images around local midnight in a region with suitable atmospheric and weather conditions.

Finally, system level integration with the attitude determination and control subsystem should be addressed. This includes fusing star tracker measurements with

gyroscope and sun sensor data through an extended Kalman filter, defining operational modes for lost-in-space and tracking, and designing fault detection and recovery strategies. In the longer term, on orbit experiments in a future INTISAT mission would provide the ultimate validation of the proposed hardware and algorithms and would enable further refinement of both the design and the processing pipeline based on real flight data.

BIBLIOGRAPHY

[1] G. Russell and S. Tomás, “Developing and testing an ultra-low-cost star tracker for attitude determinations of nanosatellites,” 2023, doi: 10.58011/469p-rx74.

[2] California Polytechnic State University, *CubeSat Design Specification*, Feb. 2022. Accessed: Oct. 09, 2024. [Online]. Available: https://static1.squarespace.com/static/5418c831e4b0fa4ecac1bacd/t/62193b7fc9e72e0053f00910/1645820809779/CDS+REV14_1+2022-02-09.pdf

[3] B. K. Malphrus, A. Freeman, R. Staehle, A. T. Klesh, and R. Walker, “4 - Interplanetary CubeSat missions,” in *Cubesat Handbook*, C. Cappelletti, S. Battistini, and B. K. Malphrus, Eds., Academic Press, 2021, pp. 85–121. doi: 10.1016/B978-0-12-817884-3.00004-7.

[4] H. Heidt, J. Puig-Suari, A. Moore, S. Nakasuka, and R. Twiggs, “CubeSat: A New Generation of Picosatellite for Education and Industry Low-Cost Space Experimentation,” *Small Satellite Conference*, Aug. 2000, [Online]. Available: https://digitalcommons.usu.edu/smallsat/2000/All2000/32

[5] D. A. Bearden, “Small-satellite costs,” in *Crosslink*, 2001, pp. 32–44. Accessed: Oct. 09, 2024. [Online]. Available: https://spacese.spacegrant.org/uploads/Costs/BeardenComplexityCrosslink.pdf

[6] T. Villela, C. A. Costa, A. M. Brandão, F. T. Bueno, and R. Leonardi, “Towards the Thousandth CubeSat: A Statistical Overview,” *International Journal of Aerospace Engineering*, vol. 2019, no. 1, p. 5063145, 2019, doi: 10.1155/2019/5063145.

[7] Teledyne Defense Electronics, “The Space COTS Dilemma.” Accessed: Oct. 09, 2024. [Online]. Available: https://www.teledynedefenseelectronics.com/e2vhrel/Documents/Teledyne%20HiRel%20-%20The%20Space%20COTS%20Dilemma%20-%20v1.01.pdf

[8] GE Power Management, “Technical Service Bulletin: Tin Whiskers in MOD10 Relays,” Mar. 27, 2000. Accessed: Oct. 09, 2024. [Online]. Available: https://nepp.nasa.gov/whisker/reference/tech_papers/2000-GE-bulletin-tin-whiskers.pdf

[9] “Mitigating and Preventing the Growth of Tin and Other Metal Whiskers on Critical Hardware,” ResearchGate. Accessed: Nov. 23, 2024. [Online]. Available: https://www.researchgate.net/publication/229021577_Mitigating_and_Preventing_the_Growth_of_Tin_and_Other_Metal_Whiskers_on_Critical_Hardware

[10] D. L. Thomsen, W. Kim, and J. W. Cutler, “Shields-1, A SmallSat Radiation Shielding Technology Demonstration,” Aug. 08, 2015. Accessed: Nov. 23, 2024. [Online]. Available: https://ntrs.nasa.gov/citations/20160006374

[11] J. Diebel, “Representing Attitude: Euler Angles, Unit Quaternions, and Rotation Vectors,” *Matrix*, vol. 58, Jan. 2006.

[12] O. Björkgren, “Implementation of FPGA-based star tracker pre-processing pipeline,” Diplomarbete, Åbo Akademi University, 2022. Accessed: Oct. 12, 2024. [Online]. Available: https://www.doria.fi/handle/10024/186226

[13] A. R. Ayal, “Star Detection Algorithm for Estcube-2 Star Tracker,” Tartu Ülikool, 2016. Accessed: Nov. 23, 2024. [Online]. Available: http://hdl.handle.net/10062/51852

[14] R. Oskars Komarovskis, “Development and Implementation of ESTCube-2 Star Tracker FPGA Design,” Master’s Degree, University of Tartu, Faculty of Science and Technology, Institute of Technology, 2024. Accessed: Nov. 23, 2024. [Online]. Available: https://www.ims.ut.ee/www-public2/at/2024/msc/atprog-magistrit55-loti.05.036-roberts-oskars-komarovskis-text-20240103.pdf

[15] “What is FOV (Field of View) | Optical Basics.” Accessed: Oct. 19, 2025. [Online]. Available: https://www.shalomeo.com/0ptical-basics-what-is-FOV%20%28Field%20of%20View%29-shalomeo.html

[16] B. P. Chandra *et al.*, “Low-cost Raspberry Pi star sensor for small satellites. II: StarberrySense flight and in-orbit performance,” *JATIS*, vol. 10, no. 2, p. 026002, Apr. 2024, doi: 10.1117/1.JATIS.10.2.026002.

[17] B. A. McBride, R. E. Smith, A. Greenberg, S. Dixon, J.-P. Muller, and J. V. Martins, “OreSat 0.5: next-generation small satellite for global cirrus cloud detection and mapping,” in *Sensors, Systems, and Next-Generation Satellites XXVIII*, SPIE, Nov. 2024, pp. 25–32. doi: 10.1117/12.3034013.

[18] “Differences between rolling shutter artifacts and motion blur - e-con Systems.” Accessed: Oct. 18, 2025. [Online]. Available: https://www.e-consystems.com/blog/camera/technology/differences-between-rolling-shutter-artifacts-and-motion-blur/

[19] “A Comprehensive Review of Satellite Orbital Placement and Coverage Optimization for Low Earth Orbit Satellite Networks: Challenges and Solutions.” Accessed: Oct. 18, 2025. [Online]. Available: https://www.mdpi.com/3457900

[20] M. Lindh, “Development and Implementation of Star Tracker Electronics,” Degree Project, KTH Royal Institute of Technology, Electrical Engineering, Space and Plasma Physics Department, 2014. Accessed: Nov. 23, 2024. [Online]. Available: https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-148955

[21] S. B. Howell, *Handbook of CCD Astronomy*, 2nd ed. in Cambridge Observing Handbooks for Research Astronomers. Cambridge: Cambridge University Press, 2006. doi: 10.1017/CBO9780511807909.

[22] B. C. P *et al.*, “Low-Cost Raspberry Pi Star Sensor for Small Satellites,” *J. Astron. Telesc. Instrum. Syst.*, vol. 8, no. 03, July 2022, doi: 10.1117/1.JATIS.8.3.036002.

[23] X. Wei *et al.*, “A star identification algorithm based on radial and dynamic cyclic features of star pattern,” *Advances in Space Research*, vol. 63, no. 7, pp. 2245–2259, Apr. 2019, doi: 10.1016/j.asr.2018.12.027.

[24] “Low Cost Star Tracker Software | T2 Portal.” Accessed: Oct. 18, 2025. [Online]. Available: https://technology.nasa.gov/patent/TOP2-265

[25] “5.0 Guidance, Navigation, and Control - NASA.” Accessed: Oct. 19, 2025. [Online]. Available: https://www.nasa.gov/smallsat-institute/sst-soa/guidance-navigation-and-control/

[26] R. Swinbank and R. James Purser, “Fibonacci grids: A novel approach to global modelling,” *Quarterly Journal of the Royal Meteorological Society*, vol. 132, no. 619, pp. 1769–1793, 2006, doi: 10.1256/qj.05.227.

[27] H. Zhao, A. Zhuang, and M. Lembeck, “A Low-Cost, Hardware-In-The-Loop Simulator Facilitating CubeSat Star Tracker Development,” *Small Satellite Conference*, Aug. 2024, [Online]. Available: https://digitalcommons.usu.edu/smallsat/2024/all2024/212

[28] “HI-1 L0 Images.” NASA, STEREO SCIENCE CENTER. [Online]. Available: https://stereo-ssc.nascom.nasa.gov/pub/ins_data/secchi/L0/a/img/hi_1/
[29] S. Pedrotty, R. Lovelace, J. Christian, D. Renshaw, and G. Quintero, “Design and Performance of an Open-Source Star Tracker Algorithm on Commercial Off-The-Shelf Cameras and Computers,” presented at the American Astronomical Society, Washington, DC, United States, Jan. 2020. Accessed: Nov. 30, 2025. [Online]. Available: https://ntrs.nasa.gov/citations/20200001376
[30] *nasa/COTS-Star-Tracker*. (Nov. 28, 2025). Python. NASA. Accessed: Nov. 30, 2025. [Online]. Available: https://github.com/nasa/COTS-Star-Tracker